DragonflyDB DashTable Architecture Breakdown

Dissection: Arpit Bhayani & Roman Gershman
Focus: Segmented Hashing & Hardware Efficiency

1. Executive Summary & Problem Statement

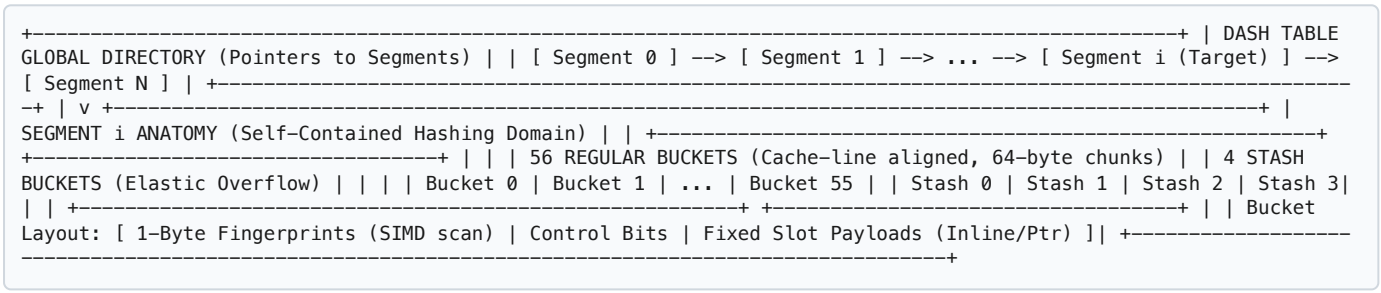
In-memory databases like Redis have traditionally relied on classic **separate chaining** (hash tables with linked lists) for their primary key-value store. While conceptually simple and robust under moderate traffic, this legacy design exhibits severe architectural limitations when scaled to modern multi-core servers with hundreds of gigabytes of RAM.

> **The Redis Memory Tax:** In Redis, every key-value pair is wrapped inside a `dictEntry` struct consuming 24 bytes of pointer metadata ($8 ext{B key} + 8 ext{B val} + 8 ext{B next}$). With jemalloc size-class quantization, each entry expends $24 ext{ to }32 ext{ bytes}$ of physical RAM solely on heap bookkeeping before storing any application payload. Storing 100 million keys wastes $2.4 ext{ to }3.2 ext{ GB}$ purely on pointers.

Furthermore, linked-list traversal triggers non-contiguous memory dereferences, causing unavoidable CPU L1/L2/L3 cache misses ("pointer chasing"). When table load factor surpasses 1.0, Redis triggers an incremental global rehash across dual hash tables (`ht[0]` and `ht[1]`), introducing latency jitter and degrading P99 SLAs under high mutation throughput.

2. The DashTable Paradigm: Hardware-Conscious Hashing

To eliminate pointer overhead, optimize for CPU cache hierarchies, and prevent latency spikes during resizing, Roman Gershman and the DragonflyDB engineering team implemented **DashTable**, heavily adapted from modern hardware-conscious indexing research (*Dash: Scalable Hashed Indices on PMem/DRAM*).



DashTable departs from node-based heap allocations by grouping keys into coarse-grained, fixed-size **Segments**. Each segment contains a matrix of cache-line-aligned **Buckets** holding contiguous slot arrays and 1-byte hash signatures (fingerprints).

Key Architectural Invariants

- **Contiguous Memory:** Buckets are strictly packed; no individual node allocations or linked-list pointers exist on the heap.
- **Hardware Cache-Line Alignment:** Buckets are sized to 64 bytes to match modern CPU cache line boundaries, ensuring an entire bucket is fetched in a single memory-bus transaction.
- **Vectorized Fingerprint Probing:** 1-byte hash fingerprints permit SIMD-accelerated parallel comparisons across all bucket slots before any key payload is inspected.

Segment Anatomy, Dual-Bucket Routing & Probing

Dissection: Arpit Bhayani & Roman Gershman
Focus: Segmented Hashing & Hardware Efficiency

3. Anatomical Breakdown of a Segment

A DashTable segment is an autonomous hashing partition. Rather than letting hash collisions spill arbitrarily across the global address space, all insertions and lookups are strictly localized within the segment:

- **56 Regular Buckets:** Form the primary hashing space for entries mapped to the segment.
- **4 Stash Buckets:** Serve as an on-segment overflow reserve to absorb localized clustering without triggering structural resizing.
- **Segment Metadata Header:** Maintains allocation bitmaps, entry counts, and a `local_depth` counter used for extendible hash splitting.

Cacheline Alignment & 1-Byte Fingerprints

Each bucket is aligned to 64 bytes (`alignas(64)`). The bucket header stores an array of 1-byte fingerprints derived from the least significant byte of the key hash:

```
// Conceptual Bucket Memory Representation
struct alignas(64) DashBucket {
    uint8_t  fingerprints[SLOTS_PER_BUCKET]; // 1-byte hash digests
    uint16_t occupancy_mask;                 // Bitmask indicating active slots
    uint16_t displacement_mask;              // Flags displaced cuckoo entries
    Payload  slots[SLOTS_PER_BUCKET];        // Embedded pointers or inline values
};
```

When searching for a key, the thread executes a vectorized instruction (e.g., SSE/AVX mask comparison) against the `fingerprints` array. If none of the 1-byte fingerprints match $ ext{hash}(key) \& 0 ext{xFF}$, the lookup aborts immediately with **zero pointer dereferences** and exactly **one cache line miss**.

4. Dual-Bucket Routing & Cuckoo Displacement

To achieve high load factors ($\gt 90\%$) without suffering from the clustering degradations of linear probing, DashTable utilizes **Dual-Bucket Routing with Stash Overflow**:

```
Key: "user:4096" --> Hash H = xxHash64("user:4096") |--> Segment Index = H >> (64 - global_depth) |--> Home Bucket 1: h1 = (H >> 8) % 56 |--> Home Bucket 2: h2 = ((H >> 8) ^ H_stride) % 56 |--> Fingerprint: fp = H & 0xFF Routing Evaluation: [Step 1] Check h1: Is any slot empty? ----------> Insert into h1 (DONE) [Step 2] Check h2: Is any slot empty? ----------> Insert into h2 (DONE) [Step 3] Cuckoo Displacement: Can an existing key in h1 be moved to its alternate home? YES: Move existing key to its h2_alt, place new key in h1 (DONE) [Step 4] Stash Overflow: Place key into one of 4 Stash Buckets in segment (DONE) [Step 5] Both regular & stash buckets full: TRIGGER SEGMENT SPLIT (Local Depth++)
```

By giving every key two candidate home buckets (h_1 and h_2) plus 4 shared stash buckets, DashTable guarantees that probe sequences never degrade into open-ended linear scans.

5. Dynamic Segment Splitting vs. Global Rehashing

When a segment fills both its 56 regular buckets and its 4 stash buckets, it can no longer accept writes. In traditional systems like Redis, table growth requires doubling the entire hash table and initiating a global incremental rehash, introducing latency spikes under write pressure.

DashTable employs **Extendible Segment Splitting**:

- **Localized Blast Radius:** Only the saturated segment is split. All other segments in the database continue servicing client reads and writes with zero interruption.
- **Binary Partitioning:** A new segment is allocated. The items in the saturated segment are rehashed using their next discriminating hash bit; approximately 50% stay in the existing segment, and 50% migrate to the new segment.
- **Directory Pointer Update:** The top-level segment directory is updated in $O(1)$ time. If the local depth exceeds global depth, the directory pointer array is doubled without copying underlying data segments.

6. Segment-Anchored O(1) Eviction Framework

Under memory pressure, an in-memory cache must evict entries. Redis uses random sampling ($N=5$ keys across the entire database), which degrades eviction quality and requires global random memory reads.

> **O(1) Segment Eviction:** When RAM is exhausted, DragonflyDB triggers eviction directly at the segment level. It uses a **2Q / LFRU algorithm** that maintains hot and cold candidate rings within the segment. The eviction victim is identified in strict $O(1)$ time without traversing external pointer lists or sampling across shards.

7. Architectural Comparison: Redis vs. DragonflyDB

Metric / Property	Redis Dictionary	DragonflyDB DashTable	Production Benefit
Memory Organization	Separate chaining (linked lists)	Segmented open addressing	Zero pointer indirection overhead
Per-Key Metadata Tax	$24 ext{ to }32 ext{ bytes}$ (`dictEntry`)	$2 ext{ to }4 ext{ bytes}$ amortized	**30% – 40% physical RAM reduction**
CPU Cache Behavior	Frequent L1/L2 cache misses	64-byte cacheline aligned	Single-cycle vector SIMD probing
Resizing Overhead	Global incremental rehash	Extendible segment split	Eliminates P99/P999 tail latency spikes
Eviction Strategy	Global sampling ($N=5$)	Segment-anchored 2Q / LFRU	Deterministic $O(1)$ cache eviction

8. First-Principles Summary & Design Lessons

- **Pointers Are a Hidden Tax:** Node-based data structures waste enormous memory footprints on metadata and heap fragmentation. Contiguous bucket layouts maximize data density.
- **Design for CPU Cache Lines:** Memory bandwidth is the ultimate performance wall in high-throughput database systems. Aligning bucket structures to 64 bytes and utilizing 1-byte fingerprints minimizes memory bus stalls.
- **Contain Dynamic Resizing:** Global table resizing inevitably degrades tail latency. Localizing growth to individual segments guarantees predictable sub-millisecond execution.